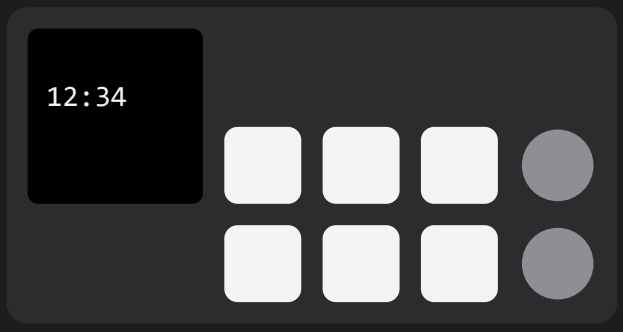


ESPDeck

Technická dokumentace – skutečný stav projektu

Stav k 09. 11. 2026

Vygenerováno z Obsidian vaultu dokumentace_md

ESP32-S3 · C++ firmware (PlatformIO) · Rust / egui aplikace

Obsah

1. Projekt **3**

ESPDeck – rozcestník . . . 3

Stav projektu . . . 4

Technická rozhodnutí . . . 5

2. Hardware **6**

HW zapojení . . . 6

Moduly a BOM . . . 8

Oživení a diagnostika . . . 9

3. Firmware **10**

Přehled firmwaru . . . 10

Obrazovky a ovládání . . . 11

Dotykové plošky . . . 12

Síťové funkce . . . 13

4. Komunikace **14**

Protokol deck–PC . . . 14

5. Aplikace **15**

Přehled aplikace . . . 15

Zkratky na ploškách . . . 16

Editor UI . . . 17

6. Grafika **18**

Animace a Display Designer . . . 18

7. Otevřené otázky **19**

Otevřené otázky a známá omezení . . . 19

8. Deník **20**

2026-11-03 . . . 20

2026-11-04 . . . 20

2026-11-08 . . . 20

2026-11-09 . . . 20

01 · PROJEKT

ESPDeck – rozcestník

aktualizováno: 2026-11-09

Ovládací panel na **ESP32-S3** (OLED 128×64, 6 dotykových plošek, rotační enkodér) a desktopová aplikace v Rustu (egui). Deck běží i bez PC: ukazuje hodiny, přes Wi-Fi ovládá Bluesound reprák a světla. Když se k němu připojí aplikace, mačkají jeho plošky klávesové zkratky na PC.

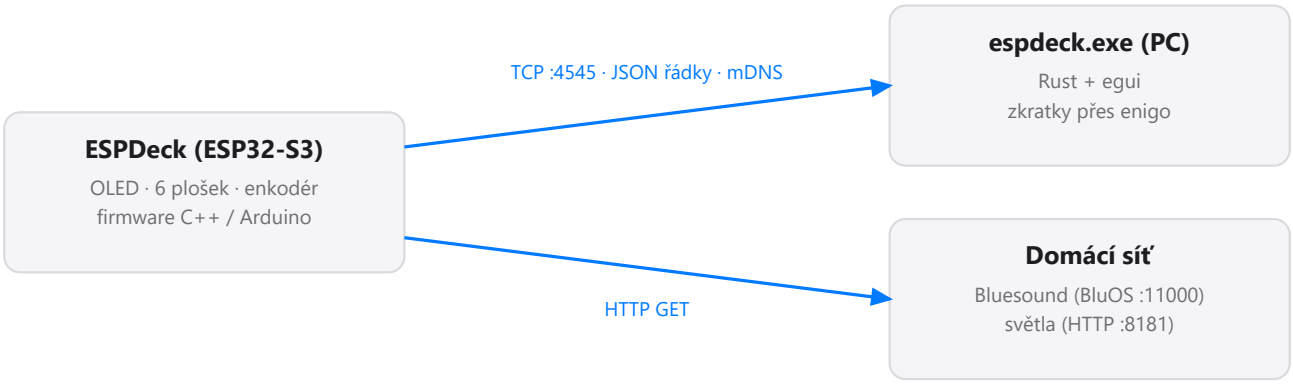
Kde jsme
Stav projektu · poslední záznam: 2026-11-09

Pravidlo vaultu
Píše se sem jen to, co **opravdu existuje v kódu** nebo bylo ověřeno na HW. Plány patří do Otevřené otázky, ne do popisu systému.

Mapa vaultu

- **Projekt:** Stav projektu · Otevřené otázky · Technická rozhodnutí
- **HW:** HW zapojení (s nákresem) · Moduly a BOM · Oživení a diagnostika
- **Firmware** (`ESP_C++_kod/test_obrazovky`): Přehled firmwaru → Obrazovky a ovládání · Dotykové plošky · Síťové funkce
- **Komunikace:** Protokol deck–PC
- **Aplikace** (`app/espdeck-ui`): Přehled aplikace → Zkratky na ploškách · Editor UI
- **Grafika:** Animace a Display Designer

Jak to spolu souvisí



Stav projektu

aktualizováno: 2026-11-09

> **Aktuálně**
> Existuje **funkční testovací firmware** (C++/Arduino, PlatformIO) na LaskaKit ESP32-S3 DevKit a **desktopová aplikace** v Rustu/egui, která se s deckem spojí přes Wi-Fi a podle dotyků mačká zkratky. Release build aplikace proběhl (8. 11. 2026). Vše je zatím **necommitnuté** v gitu.

Co funguje (podle kódu, část ověřena na HW)

Oblast	Stav	Detail
OLED SH1106 přes µŠup	● ověřeno na HW	piny našel I2C sken – HW zapojení
Enkodér + tlačítko	● ověřeno na HW	směr otočen (`ENC_REVERSE`) – Obrazovky a ovládání
6 dotykových plošek	● ověřeno na HW	kalibrace, IIR filtr, hystereze – Dotykové plošky
Obrazovky Hodiny a Nastavení	●	grafika z Display Designeru – Animace a Display Designer
Wi-Fi, NTP čas (CET/CEST)	●	Síťové funkce
Bluesound: hlasitost, Play/Pauza, čtení stavu	●	BluOS HTTP API
Světla zap/vyp (ploška 1/3)	●	HTTP GET na lokální server
Spojení s PC aplikací	●	Protokol deck–PC
Aplikace: zkratky na ploškách, živý log	●	Zkratky na ploškách
Aplikace: editor Hardware / Functions	● jen vzhled	ukázková data z návrhu – Editor UI

Co neexistuje

- Zabezpečení spojení (šifrování, párování) – spojení je otevřené TCP v LAN, viz Otevřené otázky.
- Bluetooth – přepínač na decku i v aplikaci nic nedělá.
- Ukládání nastavení na decku (jas, BT) – po restartu se vrací výchozí hodnoty.
- Konfigurace decku z aplikace – Wi-Fi a adresy se zadávají při kompilaci v `secrets.h`.
- `ESP_C++_kod/v.0.0.1/v.0.0.1.ino` – prázdná kostra.
- Testy firmwaru; Rust má pár unit testů (`model.rs`, `actions.rs`).

Co je zastaralé mimo vault

- Web (`src/data/espdeck.ts`) popisuje **klasický ESP32** s jinými piny. Platí HW zapojení.
- `První návrh dokumentace HW zapojení.pdf` (také pro klasický ESP32) byl z pracovní složky smazán; v gitu je zatím vedený jako smazaný soubor. Nahrazuje ho `ESPDeck dokumentace.pdf`.

Technická rozhodnutí

aktualizováno: 2026-11-09

Co se v projektu **skutečně používá**. Původní teoretický návrh ze 3. 11. (firmware v Rustu, Noise šifrování, párování s účtem, postcard/COBS, simulátor) se nerealizoval a byl z vaultu odstraněn.

Oblast	Použito	Kde
Deska	LaskaKit ESP32-S3-DevKit (místo klasického ESP32 z PDF)	HW zapojení
Firmware	C++ / Arduino framework, PlatformIO, U8g2, ArduinoJson	Přehled firmwaru
Souběh ve firmwaru	síť a PC spojení ve FreeRTOS úlohách na jádře 0	Přehled firmwaru
Spojení s PC	Wi-Fi, TCP :4545, JSON po řádcích, mDNS	Protokol deck–PC
Logika PC akcí	jen v aplikaci, deck posílá události	Zkratky na ploškách
Aplikace	Rust + egui/eframe 0.35, enigo, mdns-sd	Přehled aplikace
Horní lišta	převzatá z projektu OZBEAT	Editor UI
Grafika OLED	Display Designer → `assets.h`	Animace a Display Designer
Konfigurace decku	při kompilaci v `secrets.h` (mimo git)	Síťové funkce

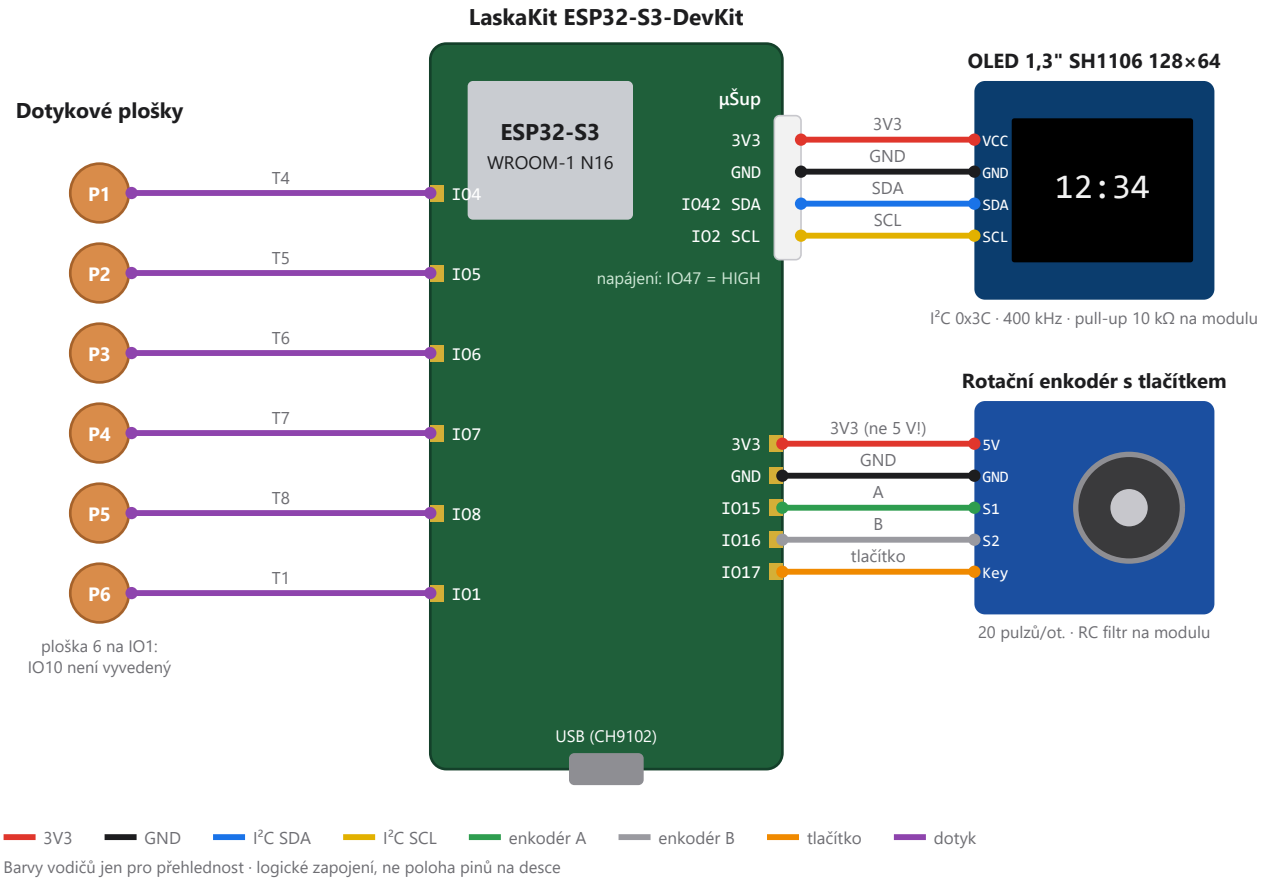
02 · HARDWARE

HW zapojení

zdroj: ESP_C++_kod/test_obrazovky/src/main.cpp · status: sestaveno, ověřeno firmwarem · aktualizováno: 2026-11-09

Skutečné zapojení prototypu podle firmwaru (Přehled firmwaru). Původní PDF „První návrh dokumentace HW zapojení“ počítalo s klasickým ESP32 (GPIO 21/22/25/26…) a **neplatí**. Moduly a kusovník: Moduly a BOM · oživení: Oživení a diagnostika.

Nákres zapojení



Logické zapojení – ukazuje, co je kam připojené, ne fyzickou polohu pinů na desce. Schéma generuje `scripts/hw_diagram.py`.

Propojení drát po drátu

3 moduly + 6 plošek, 15 vodičů. Vše na **3,3 V**, žádný převodník úrovní.

#	Modul	Pin modulu	ESP32-S3	Funkce
1	OLED	VCC	3V3 (µŠup)	napájení, spíná ho GPIO47
2	OLED	GND	GND (µŠup)	zem
3	OLED	SDA	GPIO42 (µŠup)	I²C data
4	OLED	SCL	GPIO2 (µŠup)	I²C hodiny
5	Enkodér	5V	**3V3**	napájení pull-upů a RC filtru
6	Enkodér	GND	GND	zem

#	Modul	Pin modulu	ESP32-S3	Funkce
7	Enkodér	S1	GPIO15	kanál A
8	Enkodér	S2	GPIO16	kanál B
9	Enkodér	Key	GPIO17	tlačítko (aktivní v LOW)
10	Ploška P1	–	GPIO4	dotyk T4
11	Ploška P2	–	GPIO5	dotyk T5
12	Ploška P3	–	GPIO6	dotyk T6
13	Ploška P4	–	GPIO7	dotyk T7
14	Ploška P5	–	GPIO8	dotyk T8
15	Ploška P6	–	GPIO1	dotyk T1

OLED je připojený kabelem do konektoru **µŠup** na DevKitu (vodiče 1–4 jsou v kabelu). Napájení konektoru se zapíná `GPIO47 = HIGH` – bez toho displej nesvítí ani neodpovídá na I²C.

Na co si dát pozor

- Pin „5V" enkodéru připoj na **3V3**. Pull-upy modulu by jinak daly na vstupy 5 V a ESP32-S3 není 5V tolerantní.
- Displej napájej jen z µŠup (3,3 V), ne z 5 V – pull-upy 10 kΩ na modulu vedou proti jeho VCC.

Pravidla pro piny ESP32-S3

Piny	Proč
GPIO 1–14	jen tyto umí dotyk (T1–T14); GPIO10 na DevKitu není vyvedený, proto P6 → GPIO1
GPIO 0, 3, 45, 46	strapping piny (start/boot) – nepoužívat pro plošky ani vstupy s pull-upem
GPIO 19, 20	nativní USB (D−/D+)
GPIO 43, 44	UART0 (TX/RX) přes CH9102 – nahrávání a sériová konzole
GPIO 26–32	interní SPI flash
GPIO 47	na LaskaKit DevKitu spíná napájení µŠup

Na ESP32-S3 hodnota `touchRead()` při dotyku **roste** (na klasickém ESP32 klesá) – viz Dotykové plošky.

Odchylky od původního návrhu

Oblast	Původní PDF (ESP32)	Skutečnost (ESP32-S3)
Deska	IoT ESP-WROOM-32 rev.1, CP2102	ESP32-S3-DevKit, WROOM-1 N16, CH9102
OLED	GPIO21/22, lišta 2,54 mm, 100 kHz	GPIO42/2 přes µŠup, 400 kHz
Enkodér	GPIO25/26/23, interní pull-upy vypnuté	GPIO15/16/17, interní pull-upy zapnuté, S1/S2 obráceně
Plošky	GPIO 4, 13, 14, 27, 33, 32	GPIO 4, 5, 6, 7, 8, 1
Detekce dotyku	pokles pod 80 % klidu	nárůst nad 120 % klidu

Moduly a BOM

aktualizováno: 2026-11-09

Parametry modulů OLED a enkodéru jsou převzaté z původního HW návrhu (údaje prodejce LaskaKit, ceny k 3. 11. 2026). Tyto moduly se ve skutečném zapojení použily beze změny, deska je jiná. Zapojení: HW zapojení.

Moduly

Modul	Parametr	Hodnota
LaskaKit ESP32-S3-DevKit	modul	ESP32-S3-WROOM-1 N16 (16 MB flash)
	USB-UART	CH9102
	v PlatformIO	`board = esp32-s3-devkitc-1`, `board_upload.flash_size = 16MB`
	navíc	konektor µŠup s napájením spínaným GPIO47
LaskaKit OLED 1,3" I²C	řadič / rozlišení	SH1106, 128×64, monochromatický
	adresa I²C	`0x3C` (výchozí), `0x3D` pájecím můstkem
	pull-upy	10 kΩ na SDA i SCL proti VCC displeje
	napájení	3,3–5 V, max. 100 mA (v projektu 3,3 V z µŠup)
	konektory	2× µŠup (Stemma QT / Qwiic), lišta 4 piny 2,54 mm
Rotační enkodér s tlačítkem a RC filtrem	rozlišení	20 pulzů/otáčku, neomezené otáčení
	vývody	S1, S2, Key, 5V, GND
	napájení	v projektu 3,3 V (pin „5V" na 3V3)
	dekódování	4 hrany = 1 cvak – ověřeno firmwarem

Kusovník

Ozn.	Ks	Položka	Kód	Cena/ks (Kč s DPH)
U1	1	LaskaKit ESP32-S3-DevKit	–	–
U2	1	LaskaKit OLED 128×64 1,3" I²C (bílý / modrý)	LA141084W / LA141084B	168
U3	1	Rotační enkodér s tlačítkem a RC filtrem	LA132020	48
–	1	kabel µŠup (OLED ↔ DevKit)	–	–
E1–E6	6	dotyková ploška	–	–
–	11	propojovací vodiče (enkodér 5, plošky 6)	–	–
–	1	datový USB kabel	–	–

Cena a kód desky S3 a provedení plošek (materiál, průměr, kryt, případný sériový rezistor) zatím nejsou zapsané – viz Otevřené otázky.

Oživení a diagnostika

zdroj: ESP_C++_kod/test_obrazovky (main.cpp, i2c_scan.cpp)

Jak se HW na stole oživoval a jaké nástroje na to má firmware. Zapojení: HW zapojení.

Nástroje

Nástroj	Jak spustit	Co dělá
sken pinů I²C	`pio run -e i2cscan -t upload` + `pio device monitor`	zapne µSup, zkusí všechny dvojice bezpečných GPIO jako SDA/SCL a vypíše, kde odpoví `0x3C`/`0x3D`
I²C sken při startu	běžný firmware	do sériové linky vypíše nalezené adresy, nebo „zadne (zkontroluj zapojeni displeje)“
hodnoty plošek	sériově `t`	aktuální filtrovaná hodnota a základ každé plošky, `DOTYK` u stisknuté
kalibrace plošek	sériově `c`	nová kalibrace (plošek se nedotýkat)
Wi-Fi	sériově `w`	SSID a RSSI, nebo „nepripojeno“
simulace enkodéru	sériově `n` / `p` / `o` / `l`	otočení dolů/nahoru, stisk, dlouhý stisk – bez HW

Sériová linka: 115200 Bd, port `COM8` (napevno v `platformio.ini`).

Co se při oživení zjistilo (zapsané v kódu)

- **Piny displeje** se našly skenem: SDA = GPIO42, SCL = GPIO2, adresa `0x3C`. Bez `GPIO47 = HIGH` displej neodpovídá.
- **I²C na 400 kHz** funguje s pull-upy jen na modulu (krátký kabel µSup). Na 100 kHz trval jeden snímek ~95 ms – na plynulou animaci málo.
- **Enkodér** má S1/S2 vůči A/B prohozené → `ENC_REVERSE = true`. Interní pull-upy ESP32 jsou zapnuté a nevadí.
- **Dotyk na S3:** hodnota při dotyku roste. První měření po startu jsou nesmyslně vysoká (~156 000 místo ~26 000), proto se 30 měření zahodí.
- **Wi-Fi zvyšuje šum dotyku** → kalibrace až 300 ms po startu Wi-Fi, IIR filtr a hystereze – Dotykové plošky.
- **GPIO10** na DevKitu není vyvedený → ploška 6 je na GPIO1.

Kontrolní seznam (doporučení)

1. Bez napájení: mezi 3V3 a GND není zkrat; pin „5V“ enkodéru vede na 3V3, ne na 5 V.
2. Po připojení USB: v logu `I2C zarizeni: 0x3C`.
3. Displej ukazuje hodiny, dvojtečka bliká.
4. Enkodér: na obrazovce Nastavení posouvá kurzor po jednom řádku na cvak, správným směrem.
5. Tlačítko: krátký stisk = jedna akce, dlouhý (> 600 ms) přepne obrazovku.
6. Plošky: `t` v klidu bez `DOTYK`, při doteku každé plošky `DOTYK` jen u ní.
7. S Wi-Fi: žádné falešné dotyky v klidu.

03 · FIRMWARE

Přehled firmwaru

cesta: ESP_C++_kod/test_obrazovky

Testovací firmware v **C++ / Arduino** přes **PlatformIO** (`espressif32`). Verze ohlašovaná aplikaci: `0.1.0` (`pc.cpp`).

Build

```
cd ESP_C++_kod/test_obrazovky
cp src/secrets.example.h src/secrets.h   # vyplnit Wi-Fi a adresy
pio run -t upload        # env s3 (výchozí)
pio device monitor       # 115200 Bd
pio run -e i2cscan -t upload   # jen diagnostika pinů OLED
```

Knihovny: `olikraus/U8g2` (displej), `bblanchon/ArduinoJson` (protokol s PC).

Soubory

Soubor	Co dělá
`main.cpp`	piny, obrazovky, enkodér, plošky, overlaye, smyčka – Obrazovky a ovládání, Dotykové plošky
`net.cpp/.h`	Wi-Fi, NTP, Bluesound, světla – Síťové funkce
`pc.cpp/.h`	TCP server + mDNS pro aplikaci – Protokol deck–PC
`assets.h`	bitmapy (velké číslice, ikony, malé písmo) vygenerované z Display Designeru – Animace a Display Designer
`i2c_scan.cpp`	diagnostika, kompiluje se jen s `-DI2C_PIN_SCAN`
`secrets.h`	Wi-Fi SSID/heslo, IP Bluesoundu, URL světel – **není v gitu** (`.gitignore`), vzor `secrets.example.h`

Běh

- `setup()`: zapne napájení µSup → I2C sken do logu → enkodér (přerušení) → `netBegin()` → `pcBegin()` → čas z doby kompilace → po 300 ms kalibrace plošek (až po startu Wi-Fi kvůli šumu) → OLED.
- `loop()`: sériové příkazy, plošky, tlačítko, enkodér, převzetí stavu z Bluesoundu a textu z PC, překreslení (při animaci co 15 ms, jinak co 100 ms nebo při změně).
- Síť (`net`) a PC spojení (`pc`) běží jako **samostatné FreeRTOS úlohy na jádře 0** – UI na HTTP ani TCP nikdy nečeká. Data se předávají přes proměnné chráněné `portMUX`.

Sériová konzole (115200)

`n`/`p` = enkodér dolů/nahoru · `o` = stisk · `l` = dlouhý stisk · `t` = hodnoty plošek · `c` = znovu kalibrovat plošky · `w` = stav Wi-Fi

Druhá složka

`ESP_C++_kod/v.0.0.1/v.0.0.1.ino` je zatím prázdný `setup()`/`loop()`.

Obrazovky a ovládání

zdroj: ESP_C++_kod/test_obrazovky/src/main.cpp

Obrazovky

Hodiny (výchozí)

- Velké číslice 22×40 px HH:MM, dvojtečka bliká po 500 ms.
- Dole den a datum malým písmem, např. `STREDA | 09.11`.
- Čas: do synchronizace NTP z času kompilace, pak NTP – Síťové funkce.

Nastavení – 4 řádky, vybraný je invertovaný:

Řádek	Zobrazuje	Akce stiskem
WIFI	název sítě + síla signálu (4 čárky podle RSSI), nebo `ODPOJENO`	–
BLUETOOTH	přepínač	přepne – **jen grafika, BT neexistuje**
PC	`PRIPOJENO` / `ODPOJENO` podle Protokol deck–PC	–
JAS	posuvník	zapne/vypne úpravu (knoflík bliká), krok 5 %, rozsah 5–100 %, mapováno na kontrast OLED

Nastavení se neukládá – po restartu jas 70 %, BT vypnuto.

Enkodér

Gesto	Na Hodinách	V Nastavení
otočení	hlasitost Bluesoundu ±1 (okénko s pruhem 1,2 s)	posun kurzoru / změna jasu
krátký stisk	Play/Pauza Bluesoundu + animace	akce řádku
dlouhý stisk (> 600 ms)	přepne na Nastavení	přepne na Hodiny

Každé otočení a stisk se zároveň posílá do PC aplikace (pokud je připojená). Dekódování: přerušení na obou kanálech, tabulka stavů, 4 hrany = 1 cvak. Tlačítko má debounce 30 ms.

Překryvná okénka (overlaye)

- **Hlasitost** – ikona reproduktoru (při 0 přeškrtnutá), pruh, číslo.
- **Play/Pauza** – čtverec naskočí s přestřelem, ikona se plynule přetvaruje ► ↔ || (interpolace vrcholů), po ~1,1 s zmizí.
- **Dotyk** – `DOTYK n`, nebo `SVETLA ZAP`/`SVETLA VYP` s žárovkou, nebo `BEZ WIFI`, nebo text poslaný z PC (`show`). Drží se po dobu dotyku + 700 ms.

Stav z Bluesoundu (hlasitost, hraje/nehraje) se převezme jen 3 s po poslední vlastní akci, aby hodnota „neskákala zpět".

Dotykové plošky

zdroj: ESP_C++_kod/test_obrazovky/src/main.cpp

6 plošek na GPIO 4, 5, 6, 7, 8, 1 (HW zapojení). Čte se `touchRead()` každých 10 ms.

Algoritmus

1. **Kalibrace** při startu (plošek se nedotýkat): 30 měření se zahodí (první hodnoty jsou nesmyslně vysoké, ~156k místo ~26k – ověřeno), pak průměr z 32 vzorků = základ.
2. **IIR filtr** `filt += (v - filt) * 0.3` – tlumí šum, hlavně od Wi-Fi.
3. **Hystereze:** dotyk při `filt > základ × 1,20`, uvolnění při `< základ × 1,10`.
4. **Drift:** bez dotyku se základ pomalu posouvá (`× 0.001`) za aktuální hodnotou.

Kalibrace jde zopakovat sériovým příkazem `c`, hodnoty vypsat `t`.

Co dotyk udělá

Ploška	Na decku	Do PC
1	rozsvítí světla (bez Wi-Fi ukáže `BEZ WIFI`)	`touch` pad 1
3	zhasne světla	`touch` pad 3
2, 4, 5, 6	okénko `DOTYK n`	`touch` pad n

Funkce světel je napevno v kódu (`PAD_LIGHTS_ON = 0`, `PAD_LIGHTS_OFF = 2`). Co ploška udělá na PC, rozhoduje jen aplikace – Zkratky na ploškách.

Síťové funkce

zdroj: ESP_C++_kod/test_obrazovky/src/net.cpp

Vše v `net.cpp`, běží ve vlastní FreeRTOS úloze (jádro 0, smyčka po 20 ms). UI jen zapisuje požadavky do proměnných, úloha je odešle.

Wi-Fi

- Režim STA, hostname `espdeck`, automatické znovupřipojení.
- SSID a heslo **napevno** z `secrets.h` při kompilaci.

Čas

- `configTzTime("CET-1CEST,M3.5.0,M10.5.0/3", "pool.ntp.org", "time.google.com")` – střední Evropa vč. letního času.
- Před synchronizací se použije čas kompilace.

Bluesound (BluOS HTTP API, port 11000)

Akce	Požadavek	Poznámka
hlasitost	`GET /Volume?level=N`	nejvýš 1× za 150 ms, posílá se poslední hodnota
Play/Pauza	`GET /Pause?toggle=1`	dva rychlé stisky se navzájem zruší
stav	`GET /Status`	co 5 s a hned po připojení; z XML se bere `<volume>` a `<state>` (`play`/`stream` = hraje)

IP repráku: `BLUOS_HOST` v `secrets.h`.

Světla

`GET LIGHTS_ON_URL` / `LIGHTS_OFF_URL` (v `secrets.h`, lokální server na portu 8181, endpointy `/OndraRozsvit`, `/OndraZhasni` – stejné jako IR ovladač). Spouští ploška 1 a 3 – Dotykové plošky.

Časové limity

Připojení 1,5 s, odpověď 2,5 s. Chyby se jen vypíšou do sériové linky.

04 · KOMUNIKACE

Protokol deck–PC

zdroj: ESP_C++_kod/test_obrazovky/src/pc.cpp, app/espdeck-ui/src/link.rs

Spojení

- Deck je **TCP server na portu 4545** (spustí se po připojení k Wi-Fi).
- Ohlašuje se přes **mDNS** jako služba `_espdeck._tcp`, hostname `espdeck.local`, TXT záznamy `id` (MAC z eFuse, 12 hex znaků) a `fw`.
- Aplikace hledá službu přes mDNS; když nic nenajde, zkusí `espdeck.local:4545`. Pokus o připojení každé 2 s.
- Deck drží **jedno** spojení – nové spojení nahradí staré.
- Žádné šifrování ani ověření – viz Otevřené otázky.

Formát

Jedna JSON zpráva na řádek, ukončená `\n`. Pole `t` = typ zprávy.

Deck → PC

Zpráva	Kdy
`{"t":"hello","name":"ESPDeck","id":"…","fw":"0.1.0","pads":6}`	hned po připojení
`{"t":"touch","pad":1,"ev":"tap"}`	dotyk plošky (1–6)
`{"t":"enc","d":1}`	otočení enkodéru (± cvaky)
`{"t":"btn","ev":"press"}` / `"long"`	stisk / dlouhý stisk enkodéru
`{"t":"pong"}`	odpověď na ping

PC → deck

Zpráva	Účinek
`{"t":"ping"}`	aplikace posílá co 2 s
`{"t":"show","text":"KOPIROVAT"}`	text se ukáže v okénku na OLED (max 23 znaků, převede se na velká písmena)

Hlídání spojení

- Deck: bez příchozích dat 6 s → odpojí klienta.
- Aplikace: bez příchozích dat 6 s → odpojí a hledá znovu.
- Deck neposílá nic, dokud není aplikace připojená (fronta 32 zpráv po 96 B se při odpojení vyprázdní).

Kdo co dělá

Deck posílá jen **události**. Co se má na PC stát, ví jen aplikace – Zkratky na ploškách. Enkodér a tlačítko aplikace zatím jen zapisuje do logu.

05 · APLIKACE

Přehled aplikace

cesta: app/espdeck-ui

Desktopová aplikace v **Rustu** (edice 2024) s GUI v **egui/eframe 0.35**. Binárka `espdeck` (Windows: `espdeck.exe`). Cargo workspace v kořeni repa má zatím jediný člen `app/espdeck-ui`.

Build a spuštění

```
cargo run -p espdeck-ui              # debug
cargo build --release                # target/release/espdeck.exe (bez konzole)
cargo test -p espdeck-ui
```

Závislosti

`eframe` (okno, GUI) · `enigo` (simulace kláves) · `mdns-sd` (hledání decku) · `serde`, `serde_json` · `dirs` (cesta ke konfiguraci).

Moduly

Soubor	Co dělá
`main.rs`	okno 1280×800 bez systémového rámu
`titlebar.rs`	vlastní horní lišta převzatá z OZBEAT (název vlevo, minimalizovat/maximalizovat/zavřít vpravo, změna velikosti za okraje)
`theme.rs`	barvy a písma z návrhu (Segoe UI, Consolas, Silkscreen v `assets/fonts`, licence OFL)
`app.rs`	celé okno: postranní panel, záhlaví, obrazovky – Editor UI
`link.rs`	vlákno na pozadí: mDNS, TCP spojení, ping, zpracování zpráv – Protokol deck–PC
`actions.rs`	přiřazení zkratek ploškám, jejich uložení a stisk – Zkratky na ploškách
`model.rs`, `deck.rs`, `draw.rs`	data a vykreslení decku v editoru, kreslicí pomocníci a ikony

Důležité chování

- Spojení s deckem běží ve **vlastním vlákně**, takže zkratky fungují i při minimalizovaném okně (okno ale musí běžet).
- Konfigurace: `%APPDATA%\ESPDeck\bindings.json`.

Zkratky na ploškách

zdroj: app/espdeck-ui/src/actions.rs, link.rs, app.rs (connections)

Obrazovka **Connections** v aplikaci. Každá ze 6 plošek má:

- **Shortcut** – zkratka, např. `Ctrl+C`, `Win+Shift+S`, `PlayPause`. Prázdné = nic.
- **Text on the deck** – co se po provedení ukáže na OLED (ASCII, nejlépe velkými písmeny). Prázdné = ukáže se zkratka.

Změny se ukládají hned do `%APPDATA%\ESPDeck\bindings.json`. Vedle řádku se ukazuje `OK` nebo chyba (neznámá klávesa). Dotknutá ploška se v tabulce na 700 ms zvýrazní; pod tabulkou je log posledních 14 událostí.

Výchozí přiřazení

Ploška	Zkratka	Text
1	– (světla na decku)	
2	`Ctrl+C`	KOPIROVAT
3	– (světla na decku)	
4	`Ctrl+V`	VLOZIT
5	`Win+Shift+S`	VYSTRIZEK
6	`Win+D`	PLOCHA

Názvy kláves

- Modifikátory: `Ctrl`, `Shift`, `Alt`, `Win` (`Meta`, `Super`, `Cmd`).
- Písmena a číslice: na Windows podle virtuálního kódu klávesy, takže fungují i na české klávesnici.
- `F1`–`F24`, `Enter`, `Esc`, `Tab`, `Space`, `Backspace`, `Del`, `Ins`, `Home`, `End`, `PgUp`, `PgDn`, `Up`, `Down`, `Left`, `Right`.
- Média: `PlayPause`, `Next`, `Prev`, `VolUp`, `VolDown`, `Mute`.

Stisk: modifikátory se podrží, poslední klávesa se zmáčkne, modifikátory se pustí v opačném pořadí.

Omezení

- Jen plošky – enkodér a jeho tlačítko se v aplikaci jen zapíšou do logu.
- Jedna zkratka na plošku, žádná makra ani profily.

Editor UI

zdroj: app/espdeck-ui/src/app.rs, model.rs, deck.rs

Vzhled podle `ESPDeck UI UX navrh.html` (světlé téma). macOS „semafor“ z návrhu je nahrazen lištou z OZBEAT.

Postranní panel

`Deck` · `Actions` · `Profiles` · `OLED Screens` · `Connections` · `Device` · `Settings`

Položka	Stav
Deck → Hardware	vykreslený deck na mřížce 6×3, výběr a přesun dílů, přidání/odebrání (max 8 plošek, 3 enkodéry), inspektor s piny a citlivostí
Deck → Functions	gesta a OLED odezva vybraného dílu, stránky Home/Media/Office, šablony obrazovek
Connections	● **skutečně funkční** – stav spojení, zkratky, log – Zkratky na ploškách
Actions, Profiles, OLED Screens, Device, Settings	jen text „isn't in the UI design yet“

> **Deck → Hardware/Functions jsou jen maketa**
> Data jsou **ukázková z návrhu** (`model.rs`: OLED, T1–T6, E1–E2; funkce, piny, „BLE“, „Wi-Fi“). Nic z toho se neposílá do decku ani neukládá. Přepínače Wi-Fi, BLE HID, otočení OLED, obrácení enkodéru a citlivost jsou jen v paměti okna. Piny v inspektoru odpovídají návrhu, ne HW zapojení.

06 · GRAFIKA

Animace a Display Designer

cesta: ANIMACE/

Obrazovky decku se kreslí v nástroji **Display Designer** (není součástí repa) a ukládají jako JSON do `ANIMACE/`.

Soubor	Obsah
`hodiny.v.0.0.1.json`	obrazovka Hodiny, 2 snímky (blikající dvojtečka po 500 ms)
`espdeck_nastaveni_0.0.1.json`	obrazovka Nastavení, 4 snímky

Formát: `name`, `width` 128, `height` 64, `driver` (`Ssd1306`), `tint`, `background`, `frames[]` – každý snímek má `duration_ms` a `shapes`.

Cesta do firmwaru

Z JSON návrhů vznikl `ESP_C++_kod/test_obrazovky/src/assets.h` (hlavička: „Vygenerováno z návrhů v Display Designeru. Needitovat ručně.“):

- velké číslice 0–9 (22×40 px), tečka dvojtečky,
- ikony WIFI, BT, PC, SUN,
- malé pixelové písmo (glyfy 5 px vysoké).

Rozvržení obrazovek je pak ručně přepsané v `main.cpp` – Obrazovky a ovládání. Skript, který `assets.h` generuje, v repu není.

Pozn.: návrhy uvádějí driver `Ssd1306`, skutečný displej je SH1106 (HW zapojení) – firmware používá U8g2 driver SH1106.

07 · OTEVŘENÉ OTÁZKY

Otevřené otázky a známá omezení

aktualizováno: 2026-11-09

Známá omezení současného stavu

#	Omezení	Kde
L1	**Spojení deck–PC není zabezpečené.** Kdokoli v síti se může připojit na port 4545, přebrat spojení a posílat `show` nebo číst dotyky.	Protokol deck–PC
L2	Wi-Fi heslo a adresy jsou zakompilované (`secrets.h`), změna = nový build.	Síťové funkce
L3	Bluetooth neexistuje, přepínač je jen grafika (deck i aplikace).	Obrazovky a ovládání, Editor UI
L4	Nastavení decku (jas) se neukládá.	Obrazovky a ovládání
L5	Světla jsou napevno na ploškách 1 a 3.	Dotykové plošky
L6	Enkodér a jeho tlačítko na PC nic nedělají (jen log).	Zkratky na ploškách
L7	Editor Hardware/Functions je maketa s ukázkovými daty.	Editor UI
L8	`upload_port = COM8` je napevno v `platformio.ini`.	Přehled firmwaru
L9	Web popisuje jiný HW (klasický ESP32).	HW zapojení
L10	Generátor `assets.h` z JSON není v repu.	Animace a Display Designer
L11	Provedení dotykových plošek (materiál, průměr, kryt, sériový rezistor) a kód a cena desky S3 nejsou zdokumentované.	Moduly a BOM

Otázky k rozhodnutí

#	Otázka
Q1	Jaké minimální zabezpečení spojení s PC? (např. sdílený kód/párování) – souvisí s L1
Q2	Má deck jít nastavit z aplikace (Wi-Fi, Bluesound, světla, funkce plošek) místo `secrets.h`?
Q3	Mají se části editoru Hardware/Functions napojit na skutečný deck, nebo zjednodušit?
Q4	Co bude ve `v.0.0.1` – nová čistá verze firmwaru, nebo pokračovat v `test_obrazovky`?
Q5	Aktualizovat web podle ESP32-S3?
Q6	Je Bluetooth (BLE HID) vůbec potřeba?

2026-11-03

datum: 2026-11-03

Co se udělalo

- Založen Obsidian vault `dokumentace_md`.
- Sepsán teoretický návrh (firmware v Rustu, šifrování Noise, párování s účtem, binární protokol, simulátor).

Poznámka (doplněno 9. 11.)

Nic z toho se nerealizovalo, projekt šel jinou cestou (C++ firmware, JSON přes TCP). Návrh byl 9. 11. z vaultu odstraněn – Technická rozhodnutí.

2026-11-04

datum: 2026-11-04

Co se udělalo

- Založen Cargo workspace a aplikace `app/espdeck-ui` (egui 0.35) podle `ESPDeck UI UX navrh.html`: obrazovky Deck → Hardware a Functions – Editor UI.
- Horní lišta převzatá z OZBEAT, písma Silkscreen/Segoe UI/Consolas.

Poznámky

- Data v editoru jsou ukázková z návrhu, ne z decku.

2026-11-08

datum: 2026-11-08

Zapsáno zpětně 9. 11. podle souborů a jejich časů změn.

Co se udělalo

- Dopoledne: v Display Designeru navrženy obrazovky Hodiny a Nastavení (`ANIMACE/`) – Animace a Display Designer. Založena složka `ESP_C++_kod/v.0.0.1` (prázdná kostra).
- Testovací firmware `ESP_C++_kod/test_obrazovky` na ESP32-S3: OLED, enkodér, dotykové plošky, Wi-Fi, NTP, Bluesound, světla – Přehled firmwaru.
- Piny OLED zjištěny I2C skenem, ověřen směr enkodéru a chování dotyku na S3 – HW zapojení.
- Večer: spojení deck ↔ PC přes Wi-Fi (Protokol deck–PC) a v aplikaci obrazovka Connections se zkratkami na ploškách (Zkratky na ploškách).
- Release build aplikace (`target/release/espdeck.exe`).

2026-11-09

datum: 2026-11-09

Co se udělalo

- Vault přepsán podle skutečného kódu. Odstraněny teoretické poznámky (Architektura, Bezpečnost, Komunikace, ADR-001–005, Slovník, HW přehled pro klasický ESP32).
- Nové poznámky: HW zapojení, Přehled firmwaru, Obrazovky a ovládání, Dotykové plošky, Síťové funkce, Protokol deck–PC, Přehled aplikace, Zkratky na ploškách, Editor UI, Animace a Display Designer, Technická rozhodnutí.
- Stav projektu a Otevřené otázky odpovídají současnému stavu.
- HW dokumentace pro skutečný HW (ESP32-S3): HW zapojení s nákresem (`HW/zapojeni_hw.svg`, generuje `scripts/hw_diagram.py`), Moduly a BOM, Oživení a diagnostika. Platné údaje o modulech převzaty z původního PDF.
- Z vaultu vygenerováno jedno PDF `ESPDeck dokumentace.pdf` (kořen repa) skriptem `scripts/vault_to_pdf.py`.

Další krok

- Rozhodnout otázky v Otevřené otázky (hlavně zabezpečení spojení).
- Commitnout aplikaci, firmware a vault do gitu.